AI speed. Human accountability.

CREDSHIELDS · SMART CONTRACT AUDIT

Smart Contract Audit

Surge

This report presents the findings of a smart contract security audit conducted against Surge's Lending Protocol Smart Contracts. Findings were identified through manual review of the contract source by the CredShields' security team.

0	3	11	12	8	34
CRITICAL	HIGH	MEDIUM	LOW	INFO	TOTAL FINDINGS

CredShields Technologies Pte. Ltd. · Singapore
credshields.com · info@credshields.com

CONFIDENTIAL

/ CONTENTS

Table of Contents

01 Executive Summary

02 Vulnerability Summary

03 Detailed Findings

H001	Legacy Liquidation Threshold Is Mis-Inverted, Dropping the Trigger from 120% CR to ~83% CR	HIGH
H002	Permissionless Dust Deposits Can Bloat depositorQueue and DoS All Fixed-Market Operations (Including Liquidation Settlement)	HIGH
H003	Expired loans can be neither repaid nor liquidated.	HIGH
M001	A Lender Removed From the Depositor Queue Permanently Orphans Their Residual Fixed-Market Shares	MEDIUM
M002	Cross-Market Share Movements Use a Stale markets[0] Exchange Rate	MEDIUM
M003	finalizeAuction, repayFromLiquidation and buy Bypass the Pause, In-Flight Liquidations Settle While the Protocol Is Frozen	MEDIUM
M004	Protocol Reserve Is Truncated and Destroyed Across Fixed-Market Repay and Wind-Down	MEDIUM
M005	ERC-3009 transferWithAuthorization front-running strands borrower funds and permanently DoSes gasless repay	MEDIUM
M006	reactivateAuction does not reset the Dutch-auction clock, repricing a paused auction to the 10% floor	MEDIUM
M007	Unclassified legacy positions read zero live debt, enabling free debt-wipe and blocking liquidation	MEDIUM
M008	finalizeMarketDust destroys an evicted lender's fixed-market value on market close	MEDIUM
M009	confirmWithdrawal reduces collateral without re-checking the collateral ratio	MEDIUM
M010	NFT transfer moves position control but never re-binds collateral custody or clears in-flight withdrawals	MEDIUM
M011	Pausing the AuctionHouse does not freeze the Dutch-auction clock, letting the first post-unpause buyer settle at a deeply discounted price	MEDIUM
L001	reactivateAuction can phantom-activate an auction on a healthy position, enabling a free position wipe	LOW
L002	Exposure cap is a per-borrow throttle, not a cumulative cap, over-allocating front-of-queue lenders	LOW
L003	Collateral double-purposed during the withdrawal request-to-confirm window	LOW
L004	Live auctions reprice retroactively when the admin edits discount parameters	LOW
L005	Provider-uniqueness guard is bypassable via deactivate-assign-reactivate, cross-contaminating adaptive rateAtTarget	LOW

L006	createMarket does not validate reserveRateBps against the provider's getReserveRate	LOW
L007	borrowMore and addCollateral do not check loanExpiry	LOW
L008	createMarket does not bound or cross-validate market risk parameters (permanently un-liquidatable market, mis-sized reserve, instantly-liquidatable loans)	LOW
L009	Liquidation Latch Has No Cancellation, Expiry, or Cure Path	LOW
L010	Market Provider Interest Clock Not Synchronized on Provider Swap	LOW
L011	Missing Storage Gap in `Variables` Enables Upgrade Storage Collision	LOW
L012	pendingExtension Is Never Cleared on Seizure, Freezing the Borrower's Collateral Remainder	LOW
I001	_updateTickData uses raw subtraction that underflow-reverts on a tick-data desync, enabling a liquidation/repay DoS	INFO
I002	FIFO allocation pushes liquidation losses onto the earliest-queued lenders	INFO
I003	checkHealth returns silently instead of reverting PositionHealthy	INFO
I004	Origination fee and provider LTV/reserve-rate getters are configured but never used	INFO
I005	PositionNFT.VAULT_MANAGER is mutable, contradicting the spec's "immutable" claim	INFO
I006	switchMarket never verifies the destination market is active	INFO
I007	LiquidityPool per-user borrow tracking is dead: getUserBorrowAmount always returns 0	INFO
I008	Liquidated position is left flagged isActive == true after being zeroed	INFO

Lead Reviewers & Signatories

/ 01: THE PICTURE AT A GLANCE

Executive Summary

This report summarizes the security audit conducted by CredShields for Surge's Lending Protocol Smart Contracts. Our assessment identified a total of **34 findings**, none of which were Critical. The findings include **3 High-severity issues**, 11 Medium-severity issues, 12 Low-severity issues, and 8 Informational observations.

The three High-severity findings warrant immediate attention. These include an issue where a legacy liquidation threshold is mis-inverted, drastically dropping the trigger to ~83% collateral ratio and leading to inconsistent accounting. Additionally, permissionless dust deposits could bloat the `depositorQueue`, posing a denial-of-service risk to all fixed-market operations, including liquidation settlements. Finally, a significant vulnerability was identified where expired loans can neither be repaid nor liquidated, also resulting in a denial-of-service scenario.

Upon retest, **30 of the original 34 findings were confirmed as fixed**, demonstrating progress in addressing the identified vulnerabilities. The remaining 4 findings have been acknowledged by the Surge team.

Following the audit, Surge has deployed these contracts on the Base Mainnet:

- VaultManager - 0x0D5D12de1cC71060A38F25DD9d24DA1DD6eB705a
- LiquidityPool - 0xEE755F1BbcbF6e3260469D0f473522d71d3bdDda
- PositionNFT - 0xef070cc109f9f609d7C30cd0910640da9373C640
- AuctionHouse - 0xE5Ff1E177dDE3FC33f5457855b14e6bD3B0C6566

Audited scope

Commit: https://github.com/surgecredit/surge-lending-contracts/tree/5ab8dea6ce81c5462dfb284f34f73132ec0299e6

Retested scope

Commit: https://github.com/surgecredit/surge-lending-contracts/tree/1575f98423fd4bfb8a923d7bd08d045a9bde6774

Assessment overview

This engagement comprised a comprehensive smart contract security audit of Surge's Lending Protocol. The review employed a multi-faceted approach, combining manual code inspection with automated analysis tools to identify potential vulnerabilities, economic risks, and adherence to security best practices within the codebase. The objective was to ensure the robustness, integrity, and security of the protocol's core lending functionalities, including collateral management, loan origination, liquidation mechanisms, and market operations.

The audit identified a total of 34 findings, categorized by severity: 0 Critical, 3 High, 11 Medium, 12 Low, and 8 Informational. A retest was conducted, confirming that 30 findings have been fixed, while 4 have been acknowledged. The high-severity findings highlight significant operational and economic risks. Specifically, **H001: Legacy Liquidation Threshold Is Mis-Inverted** pointed to a critical error in calculating liquidation thresholds, leading to premature or delayed liquidations. **H002: Permissionless Dust Deposits Can Bloat depositorQueue and DoS All Fixed-Market Operations** revealed a denial of service vector where malicious actors could prevent crucial protocol operations. Furthermore, **H003: Expired loans can be neither repaid nor liquidated** identified a severe DoS scenario where user funds could become permanently stuck.

Medium-severity findings primarily revolved around business logic errors, inconsistent accounting, and additional denial-of-service vectors. Issues such as **M003: finalizeAuction, repayFromLiquidation and buy Bypass the Pause**, which allowed critical operations to proceed even when the protocol was paused, introduced significant control risks. Similarly, **M004: Protocol Reserve Is Truncated and Destroyed** identified accounting discrepancies that could lead to loss of protocol funds. Other notable medium findings, like **M005: ERC-3009 transferWithAuthorization front-running strands borrower funds** and **M009: confirmWithdrawal reduces collateral without re-checking the collateral ratio**, could lead to funds being stranded or positions becoming undercollateralized.

Low and informational findings pointed to areas requiring refinement in terms of input validation, edge-case handling, and overall code quality. Issues such as **L008: createMarket does not bound or cross-validate market risk parameters** highlighted a lack of robust input validation that could lead to unstable market configurations. **L011: Missing Storage Gap in Variables Enables Upgrade Storage Collision** identified a potential issue for future contract upgrades. Informational findings, including **I001: updateTickData uses raw subtraction that underflow-reverts**, offer opportunities to enhance the protocol's resilience and maintainability, ensuring a more robust and secure environment for users and liquidity providers.

ENGAGEMENT DETAILS

CLIENT	Surge
ASSESSMENT TYPE	Smart Contract Security Audit
AUDIT	Surge Lending Protocol Contracts
REPORT DATE	07 Jul 2026
START DATE	24 Jun 2026
END DATE	13 Jul 2026
RETEST DATE	29 Jul 2026
TOTAL FINDINGS	34
REVIEWER	Shashank (shashank@credshields.com) · CredShields

/ FINDING SEVERITY DISTRIBUTION

0	3	11	12	8
CRITICAL	HIGH	MEDIUM	LOW	INFO



/ 02: AT A GLANCE

Vulnerability Summary

Every finding from this assessment, ordered by severity. Follow an ID for the full write-up.

ID	VULNERABILITY	SEVERITY	CATEGORY	STATUS
H001	Legacy Liquidation Threshold Is Mis-Inverted, Dropping the Trigger from 120% CR to ~83% CR	HIGH	Inconsistent Accounting (SCWE-010)	✓ FIXED
H002	Permissionless Dust Deposits Can Bloat depositorQueue and DoS All Fixed-Market Operations (Including Liquidation Settlement)	HIGH	Denial of Service (SCWE-087)	✓ FIXED
H003	Expired loans can be neither repaid nor liquidated.	HIGH	Denial of Service (SCWE-087)	✓ FIXED
M001	A Lender Removed From the Depositor Queue Permanently Orphans Their Residual Fixed-Market Shares	MEDIUM	Inconsistent Accounting (SCWE-010)	✓ FIXED
M002	Cross-Market Share Movements Use a Stale markets[0] Exchange Rate	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED
M003	finalizeAuction, repayFromLiquidation and buy Bypass the Pause, In-Flight Liquidations Settle While the Protocol Is Frozen	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED
M004	Protocol Reserve Is Truncated and Destroyed Across Fixed-Market Repay and Wind-Down	MEDIUM	Inconsistent Accounting (SCWE-010)	✓ FIXED
M005	ERC-3009 transferWithAuthorization front-running strands borrower funds and permanently DoSes gasless repay	MEDIUM	Denial of Service (SCWE-087)	✓ FIXED
M006	reactivateAuction does not reset the Dutch-auction clock, repricing a paused auction to the 10% floor	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED
M007	Unclassified legacy positions read zero live debt, enabling free debt-wipe and blocking liquidation	MEDIUM	Lack of Input Validation (SC05-Lack Of Input Validation)	✓ FIXED
M008	finalizeMarketDust destroys an evicted lender's fixed-market value on market close	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED
M009	confirmWithdrawal reduces collateral without re-checking the collateral ratio	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED
M010	NFT transfer moves position control but never re-binds collateral custody or clears in-flight withdrawals	MEDIUM	Business Logic (SC02-LogicErrors)	ACCEPTED RISK

ID	VULNERABILITY	SEVERITY	CATEGORY	STATUS
M011	Pausing the AuctionHouse does not freeze the Dutch-auction clock, letting the first post-unpause buyer settle at a deeply discounted price	MEDIUM	Business Logic (SC02-LogicErrors)	ACCEPTED RISK
L001	reactivateAuction can phantom-activate an auction on a healthy position, enabling a free position wipe	LOW	Lack of Input Validation (SC05-Lack Of Input Validation)	✓ FIXED
L002	Exposure cap is a per-borrow throttle, not a cumulative cap, over-allocating front-of-queue lenders	LOW	Lack of Input Validation (SC05-Lack Of Input Validation)	✓ FIXED
L003	Collateral double-purposed during the withdrawal request-to-confirm window	LOW	Business Logic (SC02-LogicErrors)	✓ FIXED
L004	Live auctions reprice retroactively when the admin edits discount parameters	LOW	Business Logic (SC02-LogicErrors)	✓ FIXED
L005	Provider-uniqueness guard is bypassable via deactivate-assign-reactivate, cross-contaminating adaptive rateAtTarget	LOW	Business Logic (SC02-LogicErrors)	✓ FIXED
L006	createMarket does not validate reserveRateBps against the provider's getReserveRate	LOW	Lack of Input Validation (SC05-Lack Of Input Validation)	✓ FIXED
L007	borrowMore and addCollateral do not check loanExpiry	LOW	Lack of Input Validation (SC05-Lack Of Input Validation)	✓ FIXED
L008	createMarket does not bound or cross-validate market risk parameters (permanently un-liquidatable market, mis-sized reserve, instantly-liquidatable loans)	LOW	Lack of Input Validation (SC05-Lack Of Input Validation)	✓ FIXED
L009	Liquidation Latch Has No Cancellation, Expiry, or Cure Path	LOW	Failure to Handle Edge Cases (SCWE-083)	ACCEPTED RISK
L010	Market Provider Interest Clock Not Synchronized on Provider Swap	LOW	Block Values as a Proxy for Time (SCWE-065)	✓ FIXED
L011	Missing Storage Gap in `Variables` Enables Upgrade Storage Collision	LOW	Storage Layout Collision on Upgrade (SCWE-099)	✓ FIXED
L012	pendingExtension Is Never Cleared on Seizure, Freezing the Borrower's Collateral Remainder	LOW	Failure to Handle Edge Cases (SCWE-083)	ACCEPTED RISK

ID	VULNERABILITY	SEVERITY	CATEGORY	STATUS
I001	_updateTickData uses raw subtraction that underflow-reverts on a tick-data desync, enabling a liquidation/repay DoS	INFO	Denial of Service (SCWE-087)	✓ FIXED
I002	FIFO allocation pushes liquidation losses onto the earliest-queued lenders	INFO	Unfair Loss Distribution (SCWE-071)	✓ FIXED
I003	checkHealth returns silently instead of reverting PositionHealthy	INFO	Code Quality	✓ FIXED
I004	Origination fee and provider LTV/reserve-rate getters are configured but never used	INFO	Code Quality	✓ FIXED
I005	PositionNFT.VAULT_MANAGER is mutable, contradicting the spec's "immutable" claim	INFO	Code Quality	✓ FIXED
I006	switchMarket never verifies the destination market is active	INFO	Lack of Input Validation (SC05-Lack Of Input Validation)	✓ FIXED
I007	LiquidityPool per-user borrow tracking is dead: getUserBorrowAmount always returns 0	INFO	Code Quality	✓ FIXED
I008	Liquidated position is left flagged isActive == true after being zeroed	INFO	Code Quality	✓ FIXED

/ 03: DETAILED FINDINGS

34 findings across the assessment.

Findings are ordered by severity.

H001 · INCONSISTENT ACCOUNTING (SCWE-010)

HIGH

FIXED

Legacy Liquidation Threshold Is Mis-Inverted, Dropping the Trigger from 120% CR to ~83% CR

DESCRIPTION

checkHealth converts a market's liquidation threshold from LTV format to collateral-ratio (CR) format with liqThreshold = (10000 * 10000) / liqThresholdBps. This inversion is correct for new markets, where getMarketLiquidationThreshold returns an LTV (e.g. 8500 = 85% LTV -> 117.6% CR). For legacy positions, however, the threshold comes from getLegacyParams(), which returns liqThreshold_ = 12000 already expressed as a 120% collateral ratio, the old protocol used these values directly as CRs (its _checkCollateralRatio compares cr < minCR_ * 1e12 with no inversion, and the sibling minCR_ = 20000 is plainly a 200% CR, not an LTV).

Because the inversion runs unconditionally, the legacy 120% CR value is treated as if it were an LTV and collapsed to (10000*10000)/12000 = 8333, i.e. an ~83.33% CR trigger. A legacy position therefore is reported as healthy all the way down through its intended 120% liquidation point and only becomes liquidatable once its collateral is worth ~83% of the debt, already insolvent.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L1000
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L1203-L1207

BUSINESS IMPACT

For any legacy position, liquidation cannot start until the collateral ratio falls below ~83%, by which point the position is already ~17% insolvent and the 5% penalty cannot recover principal. Between the intended 120% trigger and the actual 83% trigger, under-collateralized positions linger as un-liquidatable "zombies," and once they finally liquidate the shortfall is realized as bad debt socialized across all lenders via the supply-exchange-rate write-down. This is a direct loss to LPs and is triggered by ordinary BTC price declines, conditional only on legacy positions existing in the deployment (created via setLegacyPositions migration or a marketId >= marketCount).

RECOMMENDED NEXT STEP

It is suggested to apply the LTV -> CR inversion only to new-market thresholds and use the legacy value directly as a collateral ratio, since legacy parameters are already stored in CR format.

RETEST

This issue is fixed as only new markets invert LTV->CR. Legacy no longer double-inverts the threshold (value used directly as CR).

H002 · DENIAL OF SERVICE (SCWE-087)

HIGH

Permissionless Dust Deposits Can Bloat depositorQueue and DoS All Fixed-Market Operations (Including Liquidation Settlement)

✓ FIXED

DESCRIPTION

deposit() adds the caller to depositorQueue on their first deposit with no minimum amount (only amount_ == 0 reverts) and dedups by address, so every unique address that deposits any dust (1 unit of 6-decimal USDC) becomes a permanent queue entry, entries are removed only when that address fully withdraws, and there is no queue-size cap and no admin prune. An attacker controlling many sybil addresses can therefore grow the queue without bound and never withdraw, leaving it permanently bloated.

Every fixed-market operation iterates the entire queue: _calculateTotalExposureToMarket and _allocateFundsToMarket on borrow (twice), _deallocateFundsFromMarket on repay, _proportionalDeallocate on loss liquidation, and the finalizeMarketDust wind-down sweep. Even zero-exposure dust entries cost gas per iteration (a cold userExposureBps SLOAD then continue, ~2,300 gas each), so on the order of ~13,000 entries pushes a single loop past a ~30M block gas limit and borrow iterates twice. Market 0 (deposit/withdraw/variable borrow) never iterates the queue and stays functional, but all fixed-market flows revert out-of-gas. On an L2 like Base the attack is cheap (thousands of low-gas deposit txs, free sybil addresses, negligible dust), and because the attacker's entries never leave the queue, the DoS persists.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L287-L291
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L851-L864
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L876
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L998

BUSINESS IMPACT

A cheap, permissionless dust-bloat can disable all fixed-market activity. The worst consequence is not "borrows revert" but that fixed-market liquidation settlement can no longer execute:finalizeAuction->repayFromLiquidation->_deallocateFundsFromMarket/_proportionalDeallocate iterates the bloated queue and reverts out-of-gas, so underwater fixed-market positions get stuck inLiquidation and their bad debt is never realized or socialized, a fund-impacting outcome, not merely a liveness annoyance. The DoS is permanent (attacker entries never auto-remove; no admin prune).. Market 0 remains usable, so the blast radius is the fixed-market product.

RECOMMENDED NEXT STEP

It is suggested to enforce a meaningful minimum deposit to make sybil-bloat economically infeasible, and/or cap queue length and add an admin/automatic prune for stale dust entries.

RETEST

Fixed. The exit-queue/ledger walk on both settlement paths deallocateViaLedger (repay) and proportionalDeallocateViaLedger (liquidation loss) is now bounded by a per-call cap (deallocMaxIter) and fails closed with DeallocationIncomplete. The queue is kept within the cap by the DCN's off-chain per-market LP limit and minimum-allocation floor, so settlements complete within the bound.

H003 · DENIAL OF SERVICE (SCWE-087)

Expired loans can be neither repaid nor liquidated.

HIGH

✓ FIXED

DESCRIPTION

Loan expiry hard-blocks every borrower exit but is never itself a liquidation trigger, and no function can extend a loan. repayWithERC3009Single reverts once expired (if (block.timestamp > loanExpiry[nftId_]) revert LoanExpired), and withdrawCollateral / withdrawCollateralWithAuthorization revert identically. checkHealth the only liquidation entry point decides purely on the collateral ratio and never reads loanExpiry: a position with positionCR >= liqThreshold * 1e12 returns without liquidating. loanExpiry[nftId] is written only in createLoan; there is no extendExpiry/renew/setLoanExpiry anywhere.
A position that is still healthy at its expiry timestamp therefore enters a permanent dead state: the borrower cannot repay, cannot withdraw collateral, and cannot be liquidated (still healthy). Interest keeps accruing, so the CR eventually falls below the threshold and the position is force-liquidated under the complete-liquidation model (full collateral seized, penalty applied, surplus equity not returned). Expiry simultaneously blocking repayment and blocking liquidation is a permanent lock, not merely a missing convenience.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L776-L785
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L605-L610
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L1005-L1013

BUSINESS IMPACT

Any borrower whose loan reaches its deadline while healthy a routine, no-prerequisite condition permanently loses the ability to repay and reclaim collateral; the off-chain BTC is effectively frozen, and the only resolution is eventual complete liquidation, forfeiting all equity above debt plus penalty.

RECOMMENDED NEXT STEP

Make expiry an explicit, resolvable state do not let it block repayment and liquidation at the same time. Pick one or more: allow repayment after expiry (optionally with a late fee); make expiry a liquidation trigger in checkHealth (proceed to auction once block.timestamp > loanExpiry[nftId] regardless of CR); or add an admin-gated extendExpiry(nftId, newExpiry).

RETEST

This issue is fixed as loanExpiry enforcement removed and replaced by the positionExitBlock + checkDelinquency model so expired loans are repayable and seizable.

M001 · INCONSISTENT ACCOUNTING (SCWE-010)

MEDIUM

A Lender Removed From the Depositor Queue Permanently Orphans Their Residual Fixed-Market Shares

✓ FIXED

DESCRIPTION

In LiquidityPoolV3, a lender's fixed-market value is held as userPositions[user][marketId].supplyShares, and the only way those shares return to the lender is _deallocateFundsFromMarket on borrower repayment or the finalizeMarketDust close-out sweep, both of which iterate depositorQueue. withdraw, however, removes a lender from the queue based purely on market-0 state (dp.totalDepositedAssets == 0 && userActualAllocation[user][0] == 0); it never checks whether the lender still holds fixed-market shares or locked allocation. Compounding this, _deallocateFundsFromMarket subtracts the interest-inflated shareValue from the principal-only counter userActualAllocation[marketId] and, when that counter hits zero, force-zeroes userLockedAllocation, after which the user is skipped on every future deallocation (if (userLocked == 0) continue). This can leave a lender holding residual (interest-worth) fixed-market shares while their locked counter reads zero.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L373-L375
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L1001

BUSINESS IMPACT

A lender can permanently lose the residual (typically accrued-interest) portion of a fixed-market position when they are removed from the depositor queue while still holding fixed-market shares: no on-chain path can return those shares once the lender is de-queued and the market subsequently closes.

RECOMMENDED NEXT STEP

It is suggested to block queue removal in withdraw while the lender still has any fixed-market exposure, require _getUserTotalLocked(user) == 0 and zero residual fixed-market shares before calling _removeFromQueue.

RETEST

Fixed. Queue removal is gated on _getUserTotalLocked()==0, so a lender with residual fixed-market value is retained and still credited on wind-down.

M002 · BUSINESS LOGIC (SC02-LOGICERRORS)

Cross-Market Share Movements Use a Stale markets[0] Exchange Rate

MEDIUM

✓ FIXED

DESCRIPTION

Whenever value moves between a fixed market and market 0, the pool reads or writes market-0 supply shares at markets[0].supplyExchangeRate (or its totalSupplyShares/totalSupplyAssets ratio) but the operation only accrues the fixed market first, never market 0. borrow() accrues marketId_ then burns the lender's market-0 shares in _allocateFundsToMarket at the stale market-0 rate. Symmetrically, repay() accrues marketId_ then mints market-0 shares in _deallocateFundsFromMarket and _migrateProtocolSharesToVariable at the stale rate; repayFromLiquidation and the finalizeMarketDust wind-down sweep do the same. Only deposit/withdraw accrue market 0, because they operate on it directly so every fixed-market□market-0 crossing skips it and violates the pool's own accrue-first invariant.

When market 0 has un-booked interest, the stale (under-stated) rate skews the share math in both directions. On the burn side, burnVShares = toAllocate / R_stale over-burns the allocating lender's market-0 shares, transferring a slice of their pending interest to the remaining LPs. On the mint side, the more serious one, mintVShares = shareValue / R_stale over-mints market-0 shares for the deallocated party, which is classic interest front-running: shares are minted at the pre-accrual (discounted) rate, then capture pending market-0 interest the moment the market is next accrued, diluting honest market-0 LPs. The mint side is attacker-influenceable: repay is borrower-triggerable (repayWithERC3009Single -> LP.repay), deallocation is FIFO over depositorQueue (queue position is gameable with dust), and an attacker can time the repay during a market-0 quiet window to maximize R_true/R_stale, then withdraw repeatably extracting pending interest from other LPs.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L388
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L451
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L688

BUSINESS IMPACT

Un-booked market-0 interest is silently redistributed among lenders on every cross-market operation, bounded by the interest accrued in market 0 since its last direct touch. The dangerous direction is the mint side (repay/deallocation/migration/wind-down): a party can acquire market-0 shares at the stale, discounted rate and capture a slice of pending interest from honest market-0 LPs, a value leak of roughly shareValue × (R_true/R_stale − 1) per operation. It is attacker-influenceable and repeatable (a fixed-market borrower who fronts the FIFO queue as a lender can time repays during market-0 quiet windows and farm the gap), and the protocol's own reserve can be mis-credited by the same stale ratio.

RECOMMENDED NEXT STEP

Restore the accrue-first invariant for cross-market operations: accrue market 0 at the top of every entry point that subsequently allocates, deallocates, migrates protocol shares, or runs the wind-down sweep (borrow, repay, repayFromLiquidation, and finalizeMarketDust when marketId_ != 0), so all market-0 share mints/burns use the current rate rather than a stale one.

RETEST

This issue has been fixed as all cross-market share mints/burns (borrow, repay, repayFromLiquidation, dust wind-down, withdraw-queue swap) price at a freshly accrued market-0 rate.

M003 · BUSINESS LOGIC (SC02-LOGICERRORS)

MEDIUM

finalizeAuction, repayFromLiquidation and buy Bypass the Pause, In-Flight Liquidations Settle While the Protocol Is Frozen

✓ FIXED

DESCRIPTION

The protocol's pause is an incident-response control meant to halt new activity on each contract while leaving borrower exits open. Its documented frozen set explicitly lists VaultManager.finalizeAuction, repayFromLiquidation and AuctionHouse.buy as frozen, and the AuctionHouse.pause() docstring states it pauses "auction creation and purchases." In the actual code, however, neither function carries a whenNotPaused modifier: finalizeAuction is guarded only by nonReentrant plus the msg.sender == auctionHouse check, and buy is guarded only by nonReentrant.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L1042
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/AuctionHouseUpgradeable.sol#L339
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L688

BUSINESS IMPACT

The pause cannot stop in-flight liquidation settlement, defeating its incident-response purpose on the path that matters most.

RECOMMENDED NEXT STEP

It is suggested to add whenNotPaused to VaultManager.finalizeAuction, repayFromLiquidation and AuctionHouse.buy so the liquidation-settlement path is covered by the pause, matching the documented frozen set.

RETEST

Fixed. whenNotPaused was added across the settlement path (buy, finalizeAuction, createAuction, checkHealth, checkDelinquency, repayFromLiquidation).

M004 · INCONSISTENT ACCOUNTING (SCWE-010)

MEDIUM

Protocol Reserve Is Truncated and Destroyed Across Fixed-Market Repay and Wind-Down

✓ FIXED

DESCRIPTION

A fixed market's protocol reserve is held as protocolSupplyShares whose value compounds in the interest-inclusive totalSupplyAssets, but the market's totalPhysicalSupply tracks principal only (repaid fixed-market interest is never booked into it that path is gated to marketId_ == 0). On repay, repay() deallocates the LP slice first, draining totalPhysicalSupply by a principal-proportional amount, and then calls _migrateProtocolSharesToVariable, which computes the protocol's interest-inclusive tokenAmount and caps it to the now-depleted physical supply while leaving sharesToBurn unchanged. The protocol's interest above its principal-proportional physical claim is therefore silently dropped, and because shares are burned without removing matching assets, the market is left with totalSupplyShares near zero and totalSupplyAssets still positive. Since the exchange-rate refresh is gated on totalSupplyShares > 0, the market is left with a massively inflated supplyExchangeRate. When the market subsequently closes, finalizeMarketDust migrates only queued lenders and then zeroes protocolSupplyShares and totalSupplyAssets outright, destroying the stranded reserve rather than migrating it. These are two consecutive stages of one value-loss path with a single shared root cause: the migrate-cap truncates and strands the protocol interest and the finalizeMarketDust zeroing destroys whatever was stranded.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L947-L989
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L529-L605

BUSINESS IMPACT

The protocol loses a recurring slice of its earned reserve fee on fixed-market repays and wind-downs, ~9.5% of the reserve in the measured single-lender trace, up to approximately the full reserve in the limit where the LP deallocation depletes nearly all physical supply. The truncated value becomes untracked cash the protocol can no longer claim, and finalizeMarketDust destroys any residual.

RECOMMENDED NEXT STEP

Reconcile the fixed market's principal-only totalPhysicalSupply with its interest-inclusive totalSupplyAssets (so the protocol migration can never be capped away), migrate the protocol slice before the LP deallocation, and refresh supplyExchangeRate even when shares reach zero, with finalizeMarketDust migrating any residual protocol shares before zeroing as a backstop.

RETEST

This issue has been fixed as protocol slice now migrates before LP deallocation with the physical cap removed, finalizeMarketDust migrates protocol and fallback lenders before zeroing, and the liquidation loss branch caches protocolSupplyShares pre-migration and keys the write-down off the migration's returned physicalReduced, eliminating the over-reduction.

M005 · DENIAL OF SERVICE (SCWE-087)

MEDIUM

ERC-3009 transferWithAuthorization front-running strands borrower funds and permanently DoSes gasless repay

✓ FIXED

DESCRIPTION

repayWithERC3009Single pulls the borrower's funds using IERC3009(token).transferWithAuthorization(...), which validates only the from signature and ignores msg.sender. Because the authorization is not bound to the recipient, any observer of the relayer's pending transaction can extract the ERC-3009 arguments and (transferV, transferR, transferS) from calldata and call USDC.transferWithAuthorization(owner, VaultManager, amount_, ...) directly with a higher priority fee.
The front-runner's call pulls amount_ from the owner into the VaultManager and marks transferNonce_ as used. The victim's repayWithERC3009Single then reverts at the transferWithAuthorization call ("authorization is used"), rolling back nonces[owner]++ but not the already-committed token transfer. The VaultManager has no sweep/rescue and never uses receiveWithAuthorization, so the borrower's tokens (up to the full loan) are permanently stranded in the contract with their debt unreduced. Every gasless repay on a public mempool is grief-able this way, making the feature a permanent denial of service in addition to the per-victim fund loss.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L845-L856

BUSINESS IMPACT

A borrower's repayment funds (up to the full loan amount) are pulled from their wallet but never credited against their debt, leaving them stranded in the sweep-less VaultManager while the debt remains outstanding; simultaneously the entire gasless-repay feature is permanently deniable by any unprivileged front-runner.

RECOMMENDED NEXT STEP

Pull funds via receiveWithAuthorization, which enforces msg.sender == to so only the VaultManager can consume the authorization, and add it to the IERC3009 interface. As defense-in-depth, measure the received balance delta and handle the already-received case, and add an admin token-rescue for any stranded balances.

RETEST

Fixed. repayWithERC3009Single now uses receiveWithAuthorization (enforces msg.sender==to), preventing the relay front-run. Off-chain signers must sign the ReceiveWithAuthorization typehash.

M006 · BUSINESS LOGIC (SC02-LOGICERRORS)

MEDIUM

FIXED

reactivateAuction does not reset the Dutch-auction clock, repricing a paused auction to the 10% floor

DESCRIPTION

getCurrentPrice computes the Dutch decay from block.timestamp - auction.createdAt, but cancelAuction and reactivateAuction only flip auction.active and never advance createdAt to exclude the paused interval. Because cancelAuction/reactivateAuction are documented as a pause/unpause pair, resuming an auction is expected to continue near where it paused; instead the paused time counts as decay time.
If enough total time has elapsed since the original creation (roughly 114 discount intervals, well within an extra day at the 2%/15-minute default), getCurrentPrice returns the minPriceBps floor (10% of debtToPay) the instant the auction is reactivated. A liquidator then buys the entire seized collateral for ~10% of the debt, and finalizeAuction's loss branch socializes the ~90% shortfall to LPs. Compounded by cancelAuction leaving inLiquidation set, reactivation is the only way to unfreeze a cancelled position, forcing the operator through the mispriced path.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/AuctionHouseUpgradeable.sol#L318-L337
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/AuctionHouseUpgradeable.sol#L216-L233

BUSINESS IMPACT

Any auction reactivated after sufficient elapsed time prices at the 10% floor regardless of the true collateral value, letting a liquidator take the full collateral for a fraction of the debt while the LPs absorb the shortfall; the harm is reached through the intended pause/unpause operation.

RECOMMENDED NEXT STEP

On reactivateAuction, advance createdAt by the paused duration (store cancelledAt in cancelAuction, then set createdAt += block.timestamp - cancelledAt) so pricing resumes where it paused, and have cancelAuction coordinate the VaultManager's inLiquidation state so cancel/reactivate is safe.

RETEST

This issue is fixed as reactivateAuction (and cancelAuction) are removed entirely.

M007 · LACK OF INPUT VALIDATION (SC05-LACK OF INPUT VALIDATION)

MEDIUM

✓ FIXED

Unclassified legacy positions read zero live debt, enabling free debt-wipe and blocking liquidation

DESCRIPTION

_getCurrentDebtFromShares returns 0 for any position whose market is in-range (!_isLegacyMarket(marketId)) but whose positionBorrowShares == 0, ignoring the debt recorded in positionData.debtStable. This is exactly the default state of a legacy V2 position after the V3 upgrade: positionMarket defaults to 0 and positionBorrowShares defaults to 0, and once the new liquidity pool is wired with at least one market, _isLegacyMarket(0) becomes false. Until the admin includes the position in a setLegacyPositions call, its live debt reads 0 despite the full V2 debt still being owed.

Because both solvency sinks branch on the debt value, they fail open. withdrawCollateral gates its entire solvency block the newCollateralSats == 0 guard and the collateral-ratio check behind if (liveDebt > 0), and checkHealth computes the ratio against zero debt, which returns type(uint256).max and reports the position permanently healthy. Worse, repayWithERC3009Single reads actualDebt = 0, caps the repay amount to 0, refunds the borrower's tokens, and sets the stored principal to 0 erasing the recorded debt without any payment.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L1358-L1375
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L632-L646
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L999-L1008

BUSINESS IMPACT

A legacy borrower can wipe their entire on-chain debt for free via a single repayWithERC3009Single call, inflicting a direct principal loss on the liquidity pool with no admin involvement, and can subsequently request release of the full BTC collateral since the withdrawal solvency check is bypassed. The affected positions can never be liquidated because checkHealth always reports them healthy. The exposure covers every legacy position in the window between wiring the V3 pool and calling setLegacyPositions, and is permanent for any legacy position omitted from that manually-enumerated list.

RECOMMENDED NEXT STEP

Make the share-based debt path fail safe: at each site that uses borrow shares as the sole debt signal (_getCurrentDebtFromShares, repayWithERC3009Single, switchMarket, getCollateralRatio), treat a position where !_isLegacyMarket(marketId) && positionBorrowShares == 0 && positionData.debtStable > 0 as legacy (return debtStable + interest) or revert, rather than returning 0. Ideally, classify all legacy positions atomically at upgrade time via a reinitializer instead of a separate setLegacyPositions transaction, so migration completeness is no longer a prerequisite for correct accounting.

RETEST

Acknowledged.

Client Comment: all legacy positions were migrated and the migrated data has already been verified.

M008 · BUSINESS LOGIC (SC02-LOGICERRORS)

MEDIUM

finalizeMarketDust destroys an evicted lender's fixed-market value on market close

✓ FIXED

DESCRIPTION

When a fixed market's borrow side fully closes, finalizeMarketDust migrates remaining fixed-market supply back to market 0 by iterating depositorQueue, then unconditionally zeroes the market's supply totals (totalSupplyShares/totalSupplyAssets/totalPhysicalSupply = 0). An evicted lender (one removed from depositorQueue while still holding fixed-market shares) is not in the queue, so their userPositions[u][marketId].supplyShares are neither migrated nor cleared yet the market total is wiped.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L529-L610

BUSINESS IMPACT

An evicted lender's fixed-market principal and yield are permanently destroyed when the market closes, and per-position share accounting is corrupted (a position holds more shares than the market total of zero), leaving unattributed USDC in the contract.

RECOMMENDED NEXT STEP

Ensure a lender is never evicted from depositorQueue while holding fixed-market shares, and make finalizeMarketDust's migration iterate actual fixed-market share holders, reconciling totalSupplyShares against migrated shares and reverting on any unmigrated remainder before zeroing.

RETEST

Fixed. Wind-down migrates residual protocol shares into market 0 and sweeps the rounding remainder before zeroing (value-conserving).

M009 · BUSINESS LOGIC (SC02-LOGICERRORS)

MEDIUM

confirmWithdrawal reduces collateral without re-checking the collateral ratio

✓ FIXED

DESCRIPTION

Collateral withdrawal is two-step: withdrawCollateral validates the post-withdrawal collateral ratio at request time and records pendingWithdrawals; confirmWithdrawal then performs the actual on-chain collateral reduction but accrues interest without ever re-checking the collateral ratio (no oracle read, no _checkCollateralRatio). Between request and confirmation the BTC price can fall and interest can accrue.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L418-L451

BUSINESS IMPACT

An operator acting on a request that looked healthy on-chain can release collateral from a now-underwater position, extracting collateral value and pushing bad-debt risk onto LPs, with no on-chain guard or signal.

RECOMMENDED NEXT STEP

In confirmWithdrawal, re-run _checkCollateralRatio(newCollateralSats, liveDebt, _getOraclePrice(), minCR) using the accrued live debt and revert if the post-withdrawal position would be unhealthy; optionally bound the request-to-confirm delay.

RETEST

Acknowledged.
Client Comment: DCN owns the withdrawal flow and validates the collateral ratio before releasing the BTC. Once the withdrawal is completed, the BTC has already been sent to the user, so confirmWithdrawal only updates the on-chain state and does not need to re-check the collateral ratio.

M010 · BUSINESS LOGIC (SC02-LOGICERRORS)

MEDIUM

ACCEPTED RISK

NFT transfer moves position control but never re-binds collateral custody or clears in-flight withdrawals

DESCRIPTION

A position is represented by a transferable PositionNFT, and VaultManager dynamically honors the current ownerOf(nftId) for every control action (withdrawal requests, EIP-712 repay/withdraw authorizations). But two pieces of collateral-side state are never reconciled to the new owner on transfer.

First, positionBtcPubkey[nftId] the BTC address the off-chain MPC releases collateral to is written once in createLoan and has no setter; both withdrawal paths emit that original pubkey as the release target. After an NFT is transferred/sold, any withdrawal the new owner initiates releases the BTC to the original creator, not the new owner. The position is effectively non-transferable for value while the contract fully supports transfers.

Second, pendingWithdrawals[nftId] is keyed by nftId, set after an owner check, and consumed by confirmWithdrawal, which performs no ownerOf re-check. A seller can front-load a full-collateral withdrawal request (which, for a repaid or zero-live-debt position, skips the CR check entirely), then sell the NFT; the encumbrance survives the transfer, is not surfaced by getPosition, and is later confirmed against the new owner's collateral (released to the seller's pubkey).

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L204
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L648-L650
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L418-L450

BUSINESS IMPACT

A buyer of a position NFT can lose the entire collateral they paid for: either they can never realize it (withdrawals always pay the creator), or a pre-transfer pending withdrawal drains it post-sale. Realization runs through the documented off-chain BTC release plus admin confirmWithdrawal, so the on-chain state records the encumbrance and the release follows.

RECOMMENDED NEXT STEP

Reconcile collateral-side state with the current owner on transfer: clear pendingWithdrawals[nftId] (and require a fresh owner-registered positionBtcPubkey) via a PositionNFT->VaultManager transfer hook, or re-read ownerOf(nftId) in confirmWithdrawal and require it matches the owner recorded at request time. Surface pendingWithdrawals in getPosition. If positions are not meant to trade, make PositionNFT soulbound.

RETEST

Acknowledged.

Client Comment: The vault leaf commits to the original borrower's EVM address, so an NFT transfer cannot redirect BTC custody, and there is no secondary market. Keeping transfers enabled. Here 'admin' refers to the Surge DCN (Distributed Custody Network) controlled account.

M011 · BUSINESS LOGIC (SC02-LOGICERRORS)

MEDIUM

ACCEPTED RISK

Pausing the AuctionHouse does not freeze the Dutch-auction clock, letting the first post-unpause buyer settle at a deeply discounted price

DESCRIPTION

getCurrentPrice derives the live auction price purely from wall-clock time since creation: timeElapsed = block.timestamp - auction.createdAt, then decays the price by discountBpsPerInterval each discountInterval down to the minPriceBps floor. The contract stores no pausedAt, no accumulated pause duration, and no setter adjusts createdAt, so the discount accrues continuously even while the auction is paused.
buy() is gated by whenNotPaused so purchases are blocked during a pause, but the price it reads on the first buy after unpause reflects the entire elapsed time including the frozen window. A pause therefore does not freeze the auction; it silently keeps discounting it.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/3d661fb72da9f16cd43742c5b6d89e13576ff86c/src/AuctionHouseUpgradeable.sol#L269-L276

BUSINESS IMPACT

The paused window is handed to the first liquidator after unpause as a free discount. At defaults the 10% floor is reached ~28.5h after creation, so a pause of roughly a day (or an incident spanning the auction) makes the auction instantly buyable at 10% of the debt on unpause. The buyer repays only the discounted currentPrice while receiving 100% of the collateral, and finalizeAuction zeroes the full position debt regardless the shortfall becomes unrecoverable LP bad debt (up to ~90% of that position's debt).

RECOMMENDED NEXT STEP

Freeze the auction clock across pauses: record pausedAt in _pause(), accumulate totalPausedDuration in _unpause(), and compute timeElapsed = block.timestamp - auction.createdAt - totalPausedDuration in getCurrentPrice (equivalently, shift each open auction's createdAt/expiresAt forward by the pause span on unpause) so settlement resumes at the pre-pause price instead of dumping toward the floor.

RETEST

Acknowledged.
Client Comment: Accepted for the current phase, bounded by the minPriceBps price floor. We will add a paused-time exclusion to the Dutch clock in a future AuctionHouse revision.

L001 · LACK OF INPUT VALIDATION (SC05-LACK OF INPUT VALIDATION)

LOW

reactivateAuction can phantom-activate an auction on a healthy position, enabling a free position wipe

✓ FIXED

DESCRIPTION

reactivateAuction guards only on !auction.active, winnerEvm == 0, and !btcDelivered all of which are true for a zeroed / never-used auction slot. It then sets active = true with debtToPay = 0 without checking that an auction ever existed for the position, and without setting VaultManager.inLiquidation. This breaks the state-machine invariant "active auction implies position inLiquidation": a healthy position with no liquidation can be given a live, zero-priced auction. Anyone then calls buy, getCurrentPrice returns 0, and finalizeAuction books the position's real debt as a lender loss and zeroes the position.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/AuctionHouseUpgradeable.sol#L225-L233

BUSINESS IMPACT

A healthy borrower's entire collateral can be wiped for free and the debt socialized onto lenders, with no genuine liquidation required only the admin role and a zero-price buy.

RECOMMENDED NEXT STEP

In reactivateAuction, require the position is genuinely in a paused liquidation before reactivating.

RETEST

Fixed. reactivateAuction was removed, eliminating phantom-activation of a healthy position.

L002 · LACK OF INPUT VALIDATION (SC05-LACK OF INPUT VALIDATION)

LOW

Exposure cap is a per-borrow throttle, not a cumulative cap, over-allocating front-of-queue lenders

✓ FIXED

DESCRIPTION

_allocateFundsToMarket computes userCanAllocate = userMarket0Balance * userExposure / 10000 against the lender's current (already-shrunken) market-0 balance on every borrow, and userLockedAllocation is only ever incremented, never read as a ceiling. FIFO order repeatedly caps and drains the queue head.
Example: A deposits 1,000 at 50% exposure. Borrow #1 of 500 caps at 500 (A □ 500 remaining); borrow #2 of 250 caps at 250 (A □ 250); borrow #3 at 125, etc. A converges to ~100% allocated to the fixed market while an identically-configured tail lender is never touched. If that market takes a liquidation loss, A bears it on ~100% of funds they believed were 50%-capped.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L868-L943

BUSINESS IMPACT

Front-of-queue lenders' fixed-market exposure exceeds their configured limit through ordinary borrow volume, exposing them to outsized, non-consented losses on a fixed-market default, asymmetrically versus later lenders.

RECOMMENDED NEXT STEP

Cap cumulatively: require userLockedAllocation[user][marketId] + toAllocate <= stableBasis * exposure / 10000, tracking exposure against a stable basis rather than the post-burn current balance.

RETEST

This issue has been fixed.

L003 · BUSINESS LOGIC (SC02-LOGICERRORS)

LOW

Collateral double-purposed during the withdrawal request-to-confirm window

✓ FIXED

DESCRIPTION

withdrawCollateral/withdrawCollateralWithAuthorization set pendingWithdrawals[nftId] = W and emit LogCollateralWithdrawalRequested (the off-chain release trigger) without decrementing collateralSats. confirmWithdrawal's own comment states it confirms "after BTC release," so the Bitcoin leaves custody before the on-chain subtraction. In the request-to-confirm window (a full BTC settlement) the same sats are counted both as paid out off-chain and as full on-chain collateral. checkHealth is permissionless, does not guard on pendingWithdrawals > 0, values the full collateralSats, and voids the pending request before auctioning the full amount. So an adverse tick in the window auctions collateral (including W) no longer in custody (buyer shortfall / owner double-dip), or a genuinely underwater position reads healthy and escapes liquidation (bad debt).

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L588-L651
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L955-L1039

BUSINESS IMPACT

During the withdrawal window an auction can promise collateral already released off-chain (buyer shortfall or owner receiving collateral twice), or a truly underwater position can escape liquidation, in both cases producing loss borne by the buyer or the LPs.

RECOMMENDED NEXT STEP

Reserve at request time by decrementing collateralSats in withdrawCollateral and restoring it in cancelWithdrawal, or value checkHealth's CR and collateralSatsForSale on collateralSats - pendingWithdrawals[nftId].

RETEST

Acknowledged

L004 · BUSINESS LOGIC (SC02-LOGICERRORS)

LOW

Live auctions reprice retroactively when the admin edits discount parameters

✓ FIXED

DESCRIPTION

getCurrentPrice reads discountInterval, discountBpsPerInterval, and minPriceBps live from storage on every call, but the Auction struct snapshots only debtToPay, collateralSatsForSale, and createdAt. The parameter setters are admin-only with generous bounds and no "no open auctions" guard, so editing any knob instantly reprices every open auction. For example, setMinPriceBps(100) drops a live 43,200-USDC auction's floor to 432 USDC, after which a permissionless buy seizes ~$40,000 of collateral for ~$432 and finalizeAuction socializes the ~99% shortfall to LPs. Even a well-intentioned floor change retroactively harms current bidders.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/AuctionHouseUpgradeable.sol#L318-L337
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/AuctionHouseUpgradeable.sol#L192-L214

BUSINESS IMPACT

Any discount-parameter change retroactively reprices all open auctions, so a permissionless buyer can capture the resulting discount and LPs absorb the shortfall.

RECOMMENDED NEXT STEP

Snapshot minPriceBps, discountBpsPerInterval, and discountInterval into the Auction struct at createAuction and have getCurrentPrice use the snapshot.

RETEST

Fixed. Auctions now snapshot discountInterval/discountBpsPerInterval/minPriceBps at creation and getCurrentPrice reads the snapshot, so admin edits no longer reprice a live auction.

L005 · BUSINESS LOGIC (SC02-LOGICERRORS)

LOW

Provider-uniqueness guard is bypassable via deactivate-assign-reactivate, cross-contaminating adaptive rateAtTarget

✓ FIXED

DESCRIPTION

The guard preventing two markets from sharing one stateful AdaptiveMarketProvider only rejects providers held by an active market (&& markets[i].active) in both createMarket and setMarketProvider, and adminSetMarketActive re-enables a market with no uniqueness re-check. An owner can therefore deactivate market 0, assign its provider to another market (the guard skips the inactive market), then reactivate market 0, leaving two active markets sharing one provider. Because the provider keeps a single global rateAtTarget/lastRateUpdate and its getBorrowRate is state-changing, the two markets then corrupt each other's rate state and steal each other's elapsed-time drift.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L214-L251
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L1534-L1562

BUSINESS IMPACT

Two markets sharing one adaptive provider silently corrupt the interest rate for both, mispricing borrower interest and lender yield with no on-chain signal.

RECOMMENDED NEXT STEP

Re-validate provider uniqueness in adminSetMarketActive on a false-to-true transition, drop the && markets[i].active qualifier so the guard considers all markets, or key the adaptive provider's rate state per market.

RETEST

Fixed. The provider-uniqueness check now rejects a provider bound to any market (active or not), closing the deactivate/reassign/reactivate bypass.

L006 · LACK OF INPUT VALIDATION (SC05-LACK OF INPUT VALIDATION)

LOW

createMarket does not validate reserveRateBps against the provider's getReserveRate

✓ FIXED

DESCRIPTION

createMarket accepts reserveRateBps_ as a free parameter and stores it without comparing it to the configured provider's getReserveRate(), though the Market struct NatSpec indicates the two are expected to agree. Interest accrual uses the market's reserveRateBps to compute the protocol cut, while lenders and borrowers reason about the provider's advertised reserve rate, so a mismatch silently misallocates accrued interest between lenders and the protocol reserve. The values coincide at deployment only by convention; nothing enforces it, and setMarketReserveRate has the same gap. A related gap: neither createMarket nor setMarketMaxLTV enforces maxLtvBps < liquidationThresholdBps, so a misconfigured market can mint loans that are immediately liquidatable.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L214-L251

BUSINESS IMPACT

A reserve-rate mismatch mis-sizes the protocol reserve and lender returns for the life of the market, and a missing max-LTV/liquidation-threshold relational check can produce instantly-liquidatable loans.

RECOMMENDED NEXT STEP

In createMarket and setMarketReserveRate, require reserveRateBps_ == IMarketProvider(provider).getReserveRate() (or read it from the provider), and require 0 < maxLtvBps_ < liquidationThresholdBps_.

RETEST

This issue has been fixed.

L007 · LACK OF INPUT VALIDATION (SC05-LACK OF INPUT VALIDATION)

borrowMore and addCollateral do not check loanExpiry

LOW

✓ FIXED

DESCRIPTION

All borrower exits (repayWithERC3009Single, withdrawCollateral, withdrawCollateralWithAuthorization) revert LoanExpired once block.timestamp > loanExpiry[nftId]. But borrowMore and addCollateral have no expiry check and never extend loanExpiry (set once in createLoan). The admin can therefore add debt to an already-expired loan that the borrower is then unable to repay (repay is expiry-blocked), guaranteeing the position drifts into full liquidation.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L465-L470
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L220-L224

BUSINESS IMPACT

Admin-triggered but user-harming: the borrower cannot exit a loan whose debt was just increased on an expired position, so it is trapped until it liquidates.

RECOMMENDED NEXT STEP

Revert borrowMore/addCollateral when block.timestamp > loanExpiry[nftId], or extend loanExpiry on borrowMore.

RETEST

Fixed. Expiry is no longer enforced on any path (removed uniformly and governed by the delinquency model).

L008 · LACK OF INPUT VALIDATION (SC05-LACK OF INPUT VALIDATION)

LOW

FIXED

createMarket does not bound or cross-validate market risk parameters (permanently un-liquidatable market, mis-sized reserve, instantly-liquidatable loans)

DESCRIPTION

createMarket validates only provider/token/decimals and then stores maxLtvBps_, liquidationThresholdBps_, and reserveRateBps_ with no bounds and no cross-checks. Three distinct gaps follow.
(1) Un-liquidatable market (most severe): checkHealth computes liqThreshold = (10000 * 10000) / liquidationThresholdBps, so a market created with liquidationThresholdBps == 0 makes checkHealth revert (division by zero) for every position in it; maxLtvBps == 0 similarly reverts the create/borrow/withdraw paths that divide by it. There is no setMarketLiquidationThreshold setter anywhere, so the value can never be corrected and the VaultManager is non-upgradeable, so the market is permanently un-liquidatable with no recovery path and unbounded bad debt.
(2) No maxLTV-vs-liquidation-threshold relation: neither createMarket nor setMarketMaxLTV enforces maxLtvBps < liquidationThresholdBps, so a misconfigured market can mint loans that are immediately liquidatable at origination.
(3) reserveRateBps not reconciled with the provider: reserveRateBps_ is stored without comparing it to the configured provider's getReserveRate(), though the Market struct NatSpec expects them to agree. Interest accrual uses the market's reserveRateBps for the protocol cut while lenders/borrowers reason about the provider's advertised rate, so a mismatch silently mis-allocates accrued interest between lenders and the protocol reserve for the market's lifetime. setMarketReserveRate has the same gap.
All three are admin-configuration-at-creation gaps, but the unbounded and uncorrectable liquidation-threshold consequence is disproportionate and unrecoverable, which is why an explicit guard is warranted rather than relying on operational care.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L214-L251
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L1000

BUSINESS IMPACT

A single bad value at market creation can make a whole market permanently un-liquidatable (unbounded bad debt, no recovery), mint instantly-liquidatable loans (maxLTV >= liquidation threshold), or mis-size the protocol reserve and lender returns for the market's lifetime (reserveRate mismatch).

RECOMMENDED NEXT STEP

In createMarket validate 0 < liquidationThresholdBps_ <= 10000, 0 < maxLtvBps_ <= 10000, maxLtvBps_ < liquidationThresholdBps_, and reserveRateBps_ <= 10000, and require reserveRateBps_ == IMarketProvider(provider_).getReserveRate() (or read it from the provider). Apply the same bounds/relational checks in setMarketMaxLTV and setMarketReserveRate, and add a setMarketLiquidationThreshold (accruing first) so a bad value is recoverable.

RETEST

Fixed. createMarket now bounds maxLtv/liquidationThreshold/reserveRate and enforces maxLtv < liquidationThreshold; setMarketMaxLTV and the new setMarketLiquidationThreshold enforce the same relation.

L009 · FAILURE TO HANDLE EDGE CASES (SCWE-083)

Liquidation Latch Has No Cancellation, Expiry, or Cure Path

LOW

ACCEPTED RISK

DESCRIPTION

`inLiquidation[nftId]` is a one-way latch. It is set to `true` by the permissionless `checkHealth` (`VaultManagerUpgradeable.sol`) and by `checkDelinquency`, but it is cleared at exactly one site inside `finalizeAuction`, which is `onlyAuctionHouse` and reachable only through a successful `AuctionHouse.buy()`. There is no cancellation, no expiry unwind, and no borrower cure.

The cancellation surface was designed but never implemented: `LogAuctionCancelled` and `LogAuctionReactivated` are declared and never emitted, the `AuctionExpired` error is declared and never thrown, and the `pause()` docstring still claims "Admin can still cancel auctions during pause" while no such function exists in the contract.

Because of this, a transient price wick can push a position marginally under the liquidation threshold; anyone then calls the permissionless `checkHealth`, latching `inLiquidation` and opening an auction. If the price recovers and the position is healthy again, every borrower path (`addCollateral`, `repayWithERC3009Single`, `withdrawCollateral`, `borrowMore`, `switchMarket`) still reverts on the latch, so the borrower cannot cure the position. The only terminal state is a sale, so a recovered position is force-liquidated at the decayed auction price with no recourse. Separately, if the wick reflects a genuine loss and collateral falls below the auction floor, no buyer appears and the latch persists, so `finalizeMarketDust` early-returns while `marketActiveDebtCount` is non-zero and the market can never be wound down.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/66f2520/src/VaultManagerUpgradeable.sol#L1249-L1267
- https://github.com/surgecredit/surge-lending-contracts/blob/66f2520/src/AuctionHouseUpgradeable.sol#L100-L124

BUSINESS IMPACT

A position that recovers to health after a transient price wick can still be force-liquidated because there is no cancellation, expiry, or cure edge and every borrower path reverts on the latch, so the borrower loses collateral they could have retained; and if a genuinely insolvent position attracts no buyer, its market is permanently blocked from winding down because the active-debt count never reaches zero.

RECOMMENDED NEXT STEP

Introduce an admin-only auction cancellation path on the AuctionHouse, guarded so it can only run before a winner is assigned, that clears the auction and calls back into the VaultManager to clear `inLiquidation` and `liquidationDetails.isActive`. The event for it already exists. This gives the liquidation state machine an abort edge for both the recovered-position case and the no-buyer wind-down case.

RETEST

Acknowledged (by design). Here 'admin' refers to the Surge DCN (Distributed Custody Network) controlled account.

L010 · BLOCK VALUES AS A PROXY FOR TIME (SCWE-065)

Market Provider Interest Clock Not Synchronized on Provider Swap

LOW

✓ FIXED

DESCRIPTION

`setMarketProvider` (`LiquidityPoolV3.sol`) accrues interest with the outgoing provider and then assigns the incoming provider without touching its interest clock. The new provider's `lastRateUpdate` is still the value set in its constructor (`AdaptiveMarketProvider.sol`), so the first post-swap accrual computes `elapsed = block.timestamp - lastRateUpdate` across the entire deploy-to-execute gap and replays that whole dead window as a single exponential drift jump, at whatever utilisation happens to exist at the first touch.

At low utilisation this lands the rate anchor below its correct value and under-charges borrowers; at high utilisation it pins to the ceiling and rate cap. The mispricing is transient in practice: `_driftRateAtTarget` only freezes when utilisation sits exactly at target (`errWAD == 0`), so once utilisation moves off the exact-target point the anchor drifts back toward its correct level on subsequent accruals.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/66f2520/src/LiquidityPoolV3.sol#L1797-L1816
- https://github.com/surgecredit/surge-lending-contracts/blob/66f2520/src/markets/AdaptiveMarketProvider.sol#L83-L96

BUSINESS IMPACT

Following a provider swap that occurs after a governance delay, the market temporarily misprices borrow interest under-charging borrowers and forgoing lender yield at low utilisation, or over-charging at high utilisation until normal utilisation movement drifts the rate anchor back to its correct level.

RECOMMENDED NEXT STEP

Immediately after assigning the new provider, synchronize its clock so no dead window is replayed. For example, call the provider's borrow-rate function with zero liquidity, which short-circuits every drift path and returns the anchor unchanged while still writing `lastRateUpdate = block.timestamp`; alternatively, add an explicit pool-only clock-sync entry point and invoke it from `setMarketProvider`.

RETEST

Fixed. setMarketProvider now syncs the incoming provider via getBorrowRate(0,0), advancing lastRateUpdate with zero drift (c416816) exactly the recommended fix. Confirm the market-0 freshness guard can't block a legitimate swap. Pending fix code.

L011 · STORAGE LAYOUT COLLISION ON UPGRADE (SCWE-099)

LOW

Missing Storage Gap in `Variables` Enables Upgrade Storage Collision

✓ FIXED

DESCRIPTION

`Variables` (`common/Variables.sol`) declares its state variables across storage slots 0 through 12 with no reserved storage gap, while every other contract in the inheritance chain reserves a clean 50-slot module (`ConstantVariablesUpgradeable` uses 4 variables plus `uint256[46] __gap`, `HelpersUpgradeable` uses `uint256[50] __gap`, and `VaultManagerUpgradeable` uses 20 variables plus `uint256[30] __gap`). The contract literally named for storing vault state is therefore the one place where appending a variable shifts every downstream slot.

Because `Variables` sits directly before `ConstantVariablesUpgradeable` in the linearization (OpenZeppelin's `Initializable` uses ERC-7201 namespaced storage and occupies no linear slots), `ConstantVariablesUpgradeable._admin` currently lands at slot 13. A future upgrade that appends any variable to `Variables` — the natural place to add vault state — pushes `_admin`, `_positionNFT`, `_oracle`, and `_liquidityPool` each down by one slot. `ADMIN()` would then return the `PositionNFT` address, and because `onlyAdmin` resolves to `currentAdmin == address(0) ? _admin : currentAdmin`, if the admin was never rotated then no address can satisfy the modifier and the protocol is left with no functioning admin.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/66f2520/src/common/Variables.sol#L1-L73
- https://github.com/surgecredit/surge-lending-contracts/blob/66f2520/src/common/ConstantVariablesUpgradeable.sol#L108

BUSINESS IMPACT

A routine future upgrade that appends state to the `Variables` contract silently shifts the admin and configuration address slots, which can corrupt admin resolution and, if the admin was never rotated, leave the upgradeable protocol permanently without a functioning admin until recovered through the ProxyAdmin owner.

RECOMMENDED NEXT STEP

Append a storage gap to `Variables` sized to bring it to a full 50-slot module (for example `uint256[37] __gap`), matching the reservation pattern used by every other module in the inheritance chain, and add a storage-layout regression check to CI so any future layout shift fails the build before deployment.

RETEST

Fixed. Layout frozen at slots 0–12 with a no-append banner + blocking CI storage-layout check, new state into VaultManagerUpgradeable's gap sounder than a gap retrofit. Confirm the CI check is wired in.

L012 · FAILURE TO HANDLE EDGE CASES (SCWE-083)

LOW

ACCEPTED RISK

pendingExtension Is Never Cleared on Seizure, Freezing the Borrower's Collateral Remainder

DESCRIPTION

The `pendingExtension[nftId]` flag is set to true only in `requestCreditLineExtension` (`VaultManagerUpgradeable.sol`) and cleared only in the admin-gated `setPositionExitBlock`; there is no user-facing cancel. Three compounding gaps turn this into a collateral freeze after a delinquency seizure.

First, `requestCreditLineExtension` unpacks `positionData` but discards the `isActive` flag and never requires the position to be active, unlike the other position-mutating entry points which gate on it. Its eligibility check is also vacuous for any zero-debt position: with `liveDebt == 0`, `_calculateCollateralRatio` returns `type(uint256).max` (`HelpersUpgradeable.sol`), so currentLtvBps = (1e8 * 1e12) / positionCR `floors to 0 and the` require(currentLtvBps + extensionLtvBuffer <= liqLtvBps)` at always passes. The flag can therefore be set even on a closed or already-settled position.

Second, both seizure paths `checkDelinquency` and `checkHealth` clear the sibling `pendingWithdrawals` flag on seizure but never clear `pendingExtension`, and `finalizeAuction` does not clear it either. The omission is clearly unintended, since the adjacent `pendingWithdrawals` clear sits on the very same lines.

Third, both on-chain collateral-reclaim paths `withdrawCollateral` and `withdrawCollateralWithAuthorization` revert while `pendingExtension` is true.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/66f2520/src/VaultManagerUpgradeable.sol#L1337-L1366
- https://github.com/surgecredit/surge-lending-contracts/blob/66f2520/src/VaultManagerUpgradeable.sol#L1337-L1366
- https://github.com/surgecredit/surge-lending-contracts/blob/66f2520/src/VaultManagerUpgradeable.sol#L1337-L1366

BUSINESS IMPACT

A borrower's leftover collateral after a delinquency seizure is frozen on-chain and reclaimable only through a side-effecting admin action the borrower cannot compel, and the vault's global collateral totals stay inflated by the un-reclaimed remainder until then; the same freeze can be induced permissionlessly by replaying a still-valid owner extension signature onto a zero-debt position.

RECOMMENDED NEXT STEP

Clear `pendingExtension` on every seizure and close path in both branches of `checkDelinquency` and in `finalizeAuction` alongside the existing `pendingWithdrawals` clear. Require the position to be active in `requestCreditLineExtension` and reject the request when the position carries no outstanding debt. Additionally, add an owner-callable cancel that clears the flag without forcing reactivation of the position.

RETEST

Acknowledged (by design). Here 'admin' refers to the Surge DCN (Distributed Custody Network) controlled account.

I001 · DENIAL OF SERVICE (SCWE-087)

INFO

FIXED

_updateTickData uses raw subtraction that underflow-reverts on a tick-data desync, enabling a liquidation/repay DoS

DESCRIPTION

updateTickData reduces a tick's tracked debt with a raw subtraction (totalDebt -= uint256(-debtDelta)), which reverts on underflow unlike the floored x > y ? x - y : 0 pattern used everywhere else in the codebase. Its only negative-delta callers are repay and finalizeAuction. If the global tickData[tick] is ever less than the position's stored tickDebt being subtracted, the subtraction underflow-reverts and the whole repay/liquidation transaction reverts with it so the position can no longer be liquidated or repaid. The reachable desync is a migrated position whose global tickData was never initialized: fixTickDataForMigratedPositions exists precisely to back-fill it, so any legacy position omitted from that repair or double-counted by its non-idempotent re-run leaves tickData[tick] below its own tickDebt and underflow-reverts on the next repay/liquidation. (Note: tickData's stored value otherwise drives no pricing/health/liquidation decision; the tick subsystem is largely vestigial but the _updateTickData call still sits on the live repay/finalizeAuction path, so the revert is real.)

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/common/HelpersUpgradeable.sol#L227-L239

BUSINESS IMPACT

A tick-data desync (most plausibly a migrated position never initialized by fixTickDataForMigratedPositions, or double-counted by its non-idempotent re-run) can make repay and liquidation revert for the affected position, leaving its collateral and debt stuck. Conditional on the desync precondition and admin-curable by a correct migration, but the floored-subtraction fix is correct defense-in-depth regardless.

RECOMMENDED NEXT STEP

Use the codebase's floored subtraction pattern in _updateTickData (totalDebt = totalDebt > dec ? totalDebt - dec : 0), and ensure fixTickDataForMigratedPositions covers every migrated position and is idempotent.

RETEST

Fixed. _updateTickData clamps at 0 instead of underflow-reverting, removing the DoS. Note: clamping silently zeroes tick debt on a desync rather than reverting.

I002 · UNFAIR LOSS DISTRIBUTION (SCWE-071)

FIFO allocation pushes liquidation losses onto the earliest-queued lenders

INFO

✓ FIXED

DESCRIPTION

Fixed-market funds are pulled FIFO from depositorQueue, but losses are distributed only across lenders holding fixed-market shares. Two lenders with identical exposure therefore bear very different default risk based purely on deposit order, which is further scrambled by swap-and-pop removal.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L869-L944

BUSINESS IMPACT

Equal-consent lenders take wildly unequal losses determined purely by deposit ordering rather than by their exposure or balance.

RECOMMENDED NEXT STEP

Allocate and distribute losses pro-rata across all exposed lenders, weighted by min(exposure, balance), rather than strict FIFO.

RETEST

Acknowledged.
Client Comment: DCN allocation/deallocation redesign resolves the FIFO loss distribution concern.

I003 · CODE QUALITY

checkHealth returns silently instead of reverting PositionHealthy

INFO

✓ FIXED

DESCRIPTION

SPEC 6.1 state checkHealth reverts PositionHealthy when CR >= liqThreshold. In code the healthy branch emits LogHealthChecked and returns success; VaultManager__PositionHealthy = 10005 is defined but never used. A relayer/bot written to the spec expecting a revert instead gets a successful tx.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L1005-L1008

BUSINESS IMPACT

Behavioral/integration mismatch between the spec, the invariant catalog, the dead error code, and the implementation.

RECOMMENDED NEXT STEP

Either revert SurgeVaultError(VaultManager__PositionHealthy) in the healthy branch, or update the spec to document the silent-return + event behavior.

RETEST

Acknowledged..

I004 · CODE QUALITY

Origination fee and provider LTV/reserve-rate getters are configured but never used

INFO

✓ FIXED

CVSS 5.0

DESCRIPTION

Market.originationFeeBps exists and is set by createMarket, and providers expose getOriginationFee(), but borrow() transfers the full amount_ to the borrower with no fee deduction and originationFeeBps/getOriginationFee() are read in no borrow/accrual path. More broadly, the pool reads its per-market stored maxLtvBps/reserveRateBps and never consults the provider's getMaxLTV()/getReserveRate(), so the whole IMarketProvider getter surface is effectively dead and stored values can silently diverge from what the provider advertises.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L388-L440

BUSINESS IMPACT

The protocol forgoes all intended origination revenue for any market with a non-zero fee, and the provider's advertised LTV/reserve-rate parameters are silently ignored. Dead/unwired feature surface, not a fund drain.

RECOMMENDED NEXT STEP

Either deduct amount_ * originationFeeBps / 10000 (routed to protocol reserves) in borrow() and consult provider.getMaxLTV()/getReserveRate() (or assert equality at createMarket), or remove the unused parameters/getters.

RETEST

Acknowledged.
Client Comment: These are for future purpose we can ignore.

I005 · CODE QUALITY

INFO

FIXED

PositionNFT.VAULT_MANAGER is mutable, contradicting the spec's "immutable" claim

DESCRIPTION

SPEC states both PositionNFT.ADMIN and VAULT_MANAGER are immutable. In code ADMIN is immutable but VAULT_MANAGER is a plain storage variable with a live admin-only setter setVaultManager. Because mint is onlyVaultManager, the admin can repoint which contract is allowed to mint position NFTs within the trusted-admin model, but the spec's flat "immutable" claim is false and hides the lever.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/PositionNFT.sol#L49-L54

BUSINESS IMPACT

Spec-vs-code mismatch plus an undisclosed admin lever over NFT minting; trusted-actor bounded.

RECOMMENDED NEXT STEP

Either make VAULT_MANAGER immutable if that was the intent, or correct to document it as admin-settable via setVaultManager.

RETEST

This issue has been fixed by updating the docs.

I006 · LACK OF INPUT VALIDATION (SC05-LACK OF INPUT VALIDATION)

INFO

switchMarket never verifies the destination market is active

✓ FIXED

DESCRIPTION

VaultManagerUpgradeable.switchMarket moves a (zero-debt) position to newMarketId_, and its only destination validation is a staticcall to getMarketMaxLTV(newMarketId_) guarded by require(success && data.length > 0). The inline comment states "Verify new market exists and is active," but LiquidityPoolV3.getMarketMaxLTV only reverts when marketId >= marketCount and returns maxLtvBps for any existing market without consulting markets[marketId].active. An admin can therefore switch a position into a market that has been deactivated via adminSetMarketActive(mid, false). The function also does not check newMarketId_ != positionMarket[nftId_] or that the destination market's token matches the position token.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L552-L580
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L1353-L1356

BUSINESS IMPACT

An admin migration can park a position in a halted market where borrowMore reverts (!market.active); the position remains serviceable and the state is recoverable by reactivating the market, so there is no direct fund loss this is an admin footgun and a code-vs-comment mismatch (the destination-active check the comment promises does not exist).

RECOMMENDED NEXT STEP

Add an explicit destination-active check (for example, add an isMarketActive(uint256) view to the LiquidityPool and require it), and consider also requiring newMarketId_ != positionMarket[nftId_] and markets[newMarketId_].token == positionToken[nftId_].

RETEST

switchMarket now verifies the destination market is active via isMarketActive and reverts MarketNotActive.

I007 · CODE QUALITY

INFO

LiquidityPool per-user borrow tracking is dead: getUserBorrowAmount always returns 0

✓ FIXED

DESCRIPTION

borrow() increments market.totalBorrowShares but never writes userPositions[borrower][marketId].borrowShares, and no code path anywhere writes that field. getUserBorrowAmount returns position.borrowShares * borrowExchangeRate / 1e18, which is therefore always 0. The pool reports every borrower as owing nothing; the real per-NFT borrow shares live only in the VaultManager. This is benign for current on-chain accounting (the VaultManager is the source of truth) but a latent bug for any integrator or future pool-side logic that trusts the pool's per-user borrow tracking.

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L388-L444
- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/LiquidityPoolV3.sol#L1327-L1333

BUSINESS IMPACT

Any integrator or future pool-side logic relying on getUserBorrowAmount or UserPosition.borrowShares reads zero debt for every borrower.

RECOMMENDED NEXT STEP

Either populate userPositions[borrower][marketId].borrowShares in borrow/repay, or remove the dead field and view to avoid misleading integrators.

RETEST

Acknowledged.
Client Comment: It is being tracked by the VaultManager contract.

I008 · CODE QUALITY

INFO

Liquidated position is left flagged isActive == true after being zeroed

✓ FIXED

DESCRIPTION

On liquidation, finalizeAuction clears the position via positionData[nftId] = _packPositionData(0, 0, 0). Because _packPositionData hard-codes the active bit, the slot unpacks to isActive = true with zero collateral and zero debt, leaving a wiped position flagged "active-but-empty" (the NFT also remains owned by the original borrower). No direct fund loss was found: checkHealth returns max CR for zero debt so the position cannot be re-liquidated, and reuse via addCollateral/borrowMore is admin-gated and re-increments marketActiveDebtCount correctly. It is a state-hygiene defect that can mislead off-chain indexers/keepers and any future logic that treats isActive as "collateralized."

AFFECTED CODE

- https://github.com/surgecredit/surge-lending-contracts/blob/5ab8dea6ce81c5462dfb284f34f73132ec0299e6/src/VaultManagerUpgradeable.sol#L1089-L1110

BUSINESS IMPACT

Off-chain systems and any future on-chain logic that reads isActive may treat fully-liquidated, empty positions as live collateralized positions, causing incorrect accounting or display; no direct on-chain fund loss.

RECOMMENDED NEXT STEP

Mark the position fully inactive on liquidation (set positionData[nftId] = 0, or add a helper that clears the active bit) so a liquidated position no longer reports isActive == true.

RETEST

Acknowledged.

DISCLOSURE

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or issue and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

REVIEWER(S) & CONTACT

Shreyas Koli
Security Auditor II

Prasad Kuri
Security Auditor

Neel Shah
Security Auditor

LEAD REVIEWERS & REPORT SIGNATORIES



Shashank
Co-founder & CEO · CredShields

Security analyst and penetration tester with **12 years of experience** in pentesting and **5 years** in managing bug bounty programs. Listed in **Google, Facebook, Apple,** and 30+ websites' hall of fame for reporting valid security issues.

CVE-2017-7063 CVE-2017-7062 CVE-2017-2458



Aditya Dixit
Audit & Research Lead · CredShields

Senior security researcher and penetration tester with **10+ years** in cybersecurity. Led pentest teams at **Cobalt and HackerOne**; previously at Binary.com. Executed **500+ pentests** across banking, fintech, payments, healthcare, telecom, and media, covering web, mobile (iOS/Android), network, cloud, code review, and smart contracts.

OSCP CREST CPSA AWS Security Specialty AWS Cloud Practitioner Qualys VMDR